Laboratório de Programação II

Departamento de Ciência da Computação
UFJF

Exercícios

- Dado o programa abaixo, indique o que será impresso:

```
class No {
    public:
       void setA(No *no) { a = no; };
       void setB(No *no) { b = no; };
       No* getA() { return a; };
       No* getB() { return b; };
       void setInfo(int f) { info = f; };
       int getInfo() { return info; };
       No() { };
       ~No() { };
    private:
       No *a, *b;
       int info;
};

int main()
{
   No *n1 = new No(); No *n2 = new No();
   No *n3 = new No(); No *n4 = new No();

   n1->setInfo(10);
   n2->setInfo(20);
   n3->setInfo(30);
   n4->setInfo(40);

   n2->setA(n1);
   n3->setB(n1);
   n1->setB(n4);
   n4->setA(n3);
   n2->setB(n1);
   n4->setB(n2);

   n1 = n4->getA();
   n2 = n2->getB()->getB();
   n3 = n1->getB();
   n4 = n2->getA()->getB();

   cout << "Teste 1" << endl;
   cout << n1->getInfo() << " " << n2->getInfo() << " ";
   cout << n3->getInfo() << " " << n4->getInfo() << endl;

   n1 = n3->getB()->getB()->getA();
   n3 = n1->getB()->getA();
   n2 = n4->getB()->getB();
   n4 = n3->getB()->getB();

   cout << "Teste 2" << endl;
   cout << n1->getInfo() << " " << n2->getInfo() << " ";
   cout << n3->getInfo() << " " << n4->getInfo() << endl;
   delete n1;
   delete n2;
   delete n3;
   delete n4;

   return 0;
}
```

	n1->getInfo()	n2->getInfo()	n3->getInfo()	n4->getInfo()
Teste 1				
Teste 2				

Exercícios

- Considerando a necessidade de se trabalhar com cálculo vetorial em 3 dimensões, criou-se o TAD Vetor3D abaixo.
  - Desenvolver o construtor, o destrutor e as operações copia, normaliza e produtoEscalar do MI deste TAD.

```
class Vetor3D
{
   private:
      float x, y, z;

   public:
      Vetor3D(float xx, float yy, float zz); // inicializa os campos
      Vetor3D* copia(); // cria e retorna uma cópia do vetor armazenado
      void normaliza(); // normaliza o vetor (norma = raiz(x²+y²+z²))
      float produtoEscalar(Vetor3D *v2); // calcula e retorna o produto
                        // escalar do vetor armazenado com um outro vetor
                        // passado como parâmetro (prod = x1*x2+y1*y2+z1*z2)
      ~Vetor3D();
};
```

Exercícios

- Considere o TAD TMatrícula para armazenar os dados de matrícula de um aluno em um dado semestre. Este TAD deve armazenar o nome do aluno, o número de matrícula e o código das disciplinas nas quais o aluno está matriculado. O aluno pode se matricular em, no máximo, 15 disciplinas no semestre. Sendo assim, deve-se utilizar um vetor para armazenar os códigos das matrículas em cada disciplina. O TAD deve conter métodos para: matricular o aluno em uma disciplina; imprimir as disciplinas nas quais o aluno está matriculado; e desmatricular o aluno de alguma disciplina. Atenção: o TAD TMatrícula guarda a matrícula de um único aluno.
  a) Criar a definição do TAD Matrícula (TMatricula.h), baseado na descrição dada acima;
  b) Implementar o método para matricular o aluno em uma disciplina dado seu código;
  c) Implementar o método para imprimir o código de todas as disciplinas nas quais o aluno está matriculado;
  d) Implementar o método para desmatricular o aluno de alguma disciplina (dado seu código);
  e) Considere que precisamos alterar o TAD TMatrícula de forma que o aluno possa se matricular em infinitas disciplinas. Mostre como deveria ser o arquivo de definição (.h) para permitir tal alteração.